Assignment 1: Branch Predictor Implementation

Pentium M-Inspired Hybrid Branch Predictor — Main Assignment and Extra Credit

Student	Sahasra Boggili
PeopleSoft ID	24954536
Course	Computer Architecture
Date	October 2026

Executive summary

This project implements a Pentium M-inspired hybrid branch direction predictor in the supplied CBP-2 infrastructure. A 4,096-entry bimodal table is paired with a tagged global predictor containing 512 sets and four ways per set. The implementation uses a 15-bit global outcome-history register to distinguish correlated branch behavior.

All 20 provided compressed benchmark traces were run with the main predictor and an additional extra-credit prototype. The main predictor achieved 5.848 average direction MPKI. The extra-credit prototype achieved 5.914 average direction MPKI and 11.777 average target MPKI. These are unweighted averages of per-trace results.

The handout's Appendix B miniature worked example was independently replayed: all 22 predictions, 22 global hit/miss decisions and 22 history-register values matched. That replay is a separate reference model, not a direct test of the production C++ class under the Appendix B configuration.

Scope and technical integrity

The required direction predictor and benchmark evaluation are distinct from the extra-credit complete-unit prototype. The latter implements loop, BTB, indirect BTB and PIR-style structures but cannot be certified as an exact reproduction of proprietary Intel hardware behavior. This report reports measured outcomes and explicitly identifies approximations.

Report map

Sections 1–3 describe the problem, architecture and state. Sections 4–6 explain prediction, update and extra-credit mechanisms. Sections 7–10 document the evaluation, measured results and independent Appendix B verification. Sections 11–12 discuss engineering trade-offs, reproducibility, limitations and references.

1. Background and objectives

A pipelined processor must fetch instructions before the outcomes of some branches are known. Incorrect predictions can force speculative instructions to be discarded, wasting execution resources and cycles. A branch predictor therefore guesses whether a conditional branch will be taken and, for target prediction, where execution will continue.

1.1 Direction versus target prediction

Direction prediction estimates taken/not-taken for a conditional branch. Target prediction estimates the destination address when control transfers. The required assignment focuses on direction; the optional complete-unit prototype also attempts targets. Direction MPKI and target MPKI are separate measures and should not be treated as interchangeable.

1.2 Why combine bimodal and global history?

A bimodal predictor learns whether each static branch tends to be taken. It is simple and often effective for strongly biased branches, but cannot distinguish outcomes that depend on preceding control flow. A history-sensitive global predictor can distinguish different recent outcome patterns. Combining them allows the tagged global table to override the inexpensive bimodal prediction only when a matching global entry exists.

1.3 Assignment scope

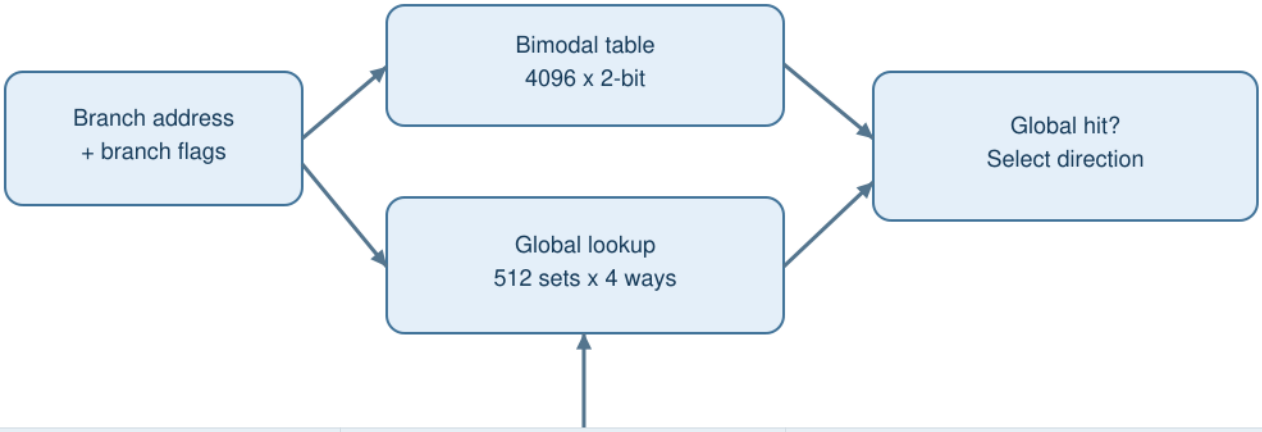
The handout describes a Pentium M-inspired bimodal/global predictor. A two-way global structure is acceptable for the required work; this implementation uses four ways. A full Pentium M-like prediction unit adds loop prediction, direct and indirect target buffers, and a path-information register. These are treated as an extra-credit approximation.

2. Implementation environment

The code runs in the supplied CBP-2 simulator. The predictor interface provides a branch address, branch flags and the resolved outcome and target through prediction and update callbacks. The same bundled traces are used for both implementations to permit like-for-like comparison.

The principal implementation resides in src/my_predictor.h, which defines the main pm_predictor and optional cpm_predictor classes. The simulator driver supplies branch records and counts prediction errors; the report does not claim to have measured execution-time speedup on physical hardware.

3. Main hybrid predictor architecture



Component	Organization	Function
Bimodal	4096 × 2-bit counters	Default direction prediction
Global	512 sets × 4 ways	Tagged, history-sensitive override
History	15-bit GHR	Recent conditional outcomes

15-bit GHR XOR address

Global table 512 sets × 4 ways = 2,048 entries Valid, 6-bit tag, 2-bit counter, LRU age

History 15-bit GHR Recent conditional outcomes

Per-prediction record pm_update Indices, tag, hit way and fallback prediction

3.2 Lookup equations

bimodal_index = address & 0xFFF hash = (address ^ GHR) & 0x7FFF global_set = hash >> 6 global_tag = hash & 0x3F

The 15-bit hash is divided into nine set-selection bits and six tag bits. For each lookup, up to four valid tags are compared in the selected set. The main predictor's unconditional branches are predicted taken.

3.3 Storage estimate

The logical main predictor requires 4,096 × 2 = 8,192 bits for bimodal counters. The global structure contains 2,048 entries × (1 valid + 6 tag + 2 counter + 2 LRU age) = 22,528 bits. Adding 15 history bits gives 30,735 bits ≈ 3.75 KiB. This is an idealized logical estimate; actual C++ structs may occupy more memory because of padding.

4. Direction prediction algorithm

4.1 Prediction steps

Step 1. Inspect the branch flags. If the branch is unconditional, predict taken. For a conditional branch, use the lower 12 address bits to read the bimodal counter.

Step 2. XOR the address with the current 15-bit global history, mask to 15 bits, and split the result into a 9-bit set index and 6-bit tag.

Step 3. Search the four ways in the selected global set. If a valid entry has the computed tag, use its counter. If there is no matching entry, use the bimodal counter.

Step 4. Store the bimodal index, global set/tag, hit status and selected way in the update object so that the later resolution can train the correct state.

4.2 Counter interpretation

Counter	Direction	Taken update	Not-taken update
0	NT	1	0
1	NT	2	0
2	T	3	1
3	T	3	2

4.3 Worked lookup example (illustrative, not benchmark evidence)

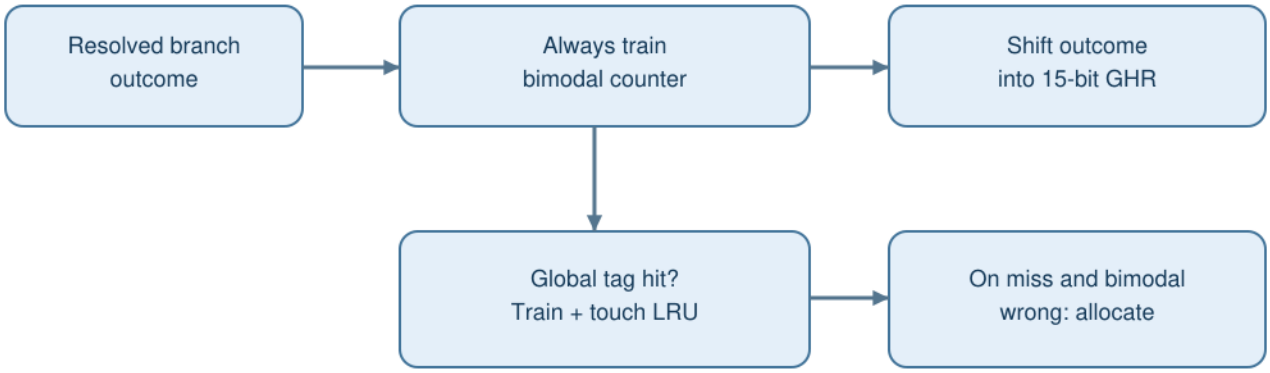
Suppose the branch address has low 15 bits equal to 0x1234 and the current GHR is 0x002A. The 15-bit XOR hash is 0x121E; therefore the global set is 72 and the tag is 30. The bimodal index is 0x234 = 564. A matching valid global tag 30 in set 72 overrides the bimodal counter; otherwise the bimodal counter 564 determines the prediction.

0x1234 XOR 0x002A = 0x121E 0x121E >> 6 = 72 0x121E & 0x3F = 30

4.4 Prediction priority

The global predictor is not consulted as a second vote. A hit has priority regardless of whether the bimodal counter agrees. On a miss, the global structure has no applicable counter and the bimodal prediction is used. This distinction matters for understanding both allocation and the Appendix B example.

5. Training, allocation and replacement



5.1 Update policy

On every resolved conditional branch, the bimodal counter is incremented for taken or decremented for not-taken, with saturation. On a global hit, the matching global counter is also trained and its way becomes most-recently-used. On a global miss, an entry is allocated only if the bimodal prediction was incorrect. The new counter is initialized weakly toward the actual outcome.

5.2 Why allocate selectively?

Branches that are already easy for the bimodal predictor need not consume scarce tagged-global capacity. Allocating only when the bimodal prediction fails reserves entries for more difficult, history-dependent branches. This is a design rationale, not a claim that a controlled allocation-policy experiment was performed in this project.

5.3 Four-way least-recently-used replacement

Each way maintains an age from zero (most recently used) to three (least recently used). When a way is touched, ways younger than its previous age become one step older, and the touched way becomes age zero. On allocation, an invalid way is used first; otherwise the way with age three is replaced.

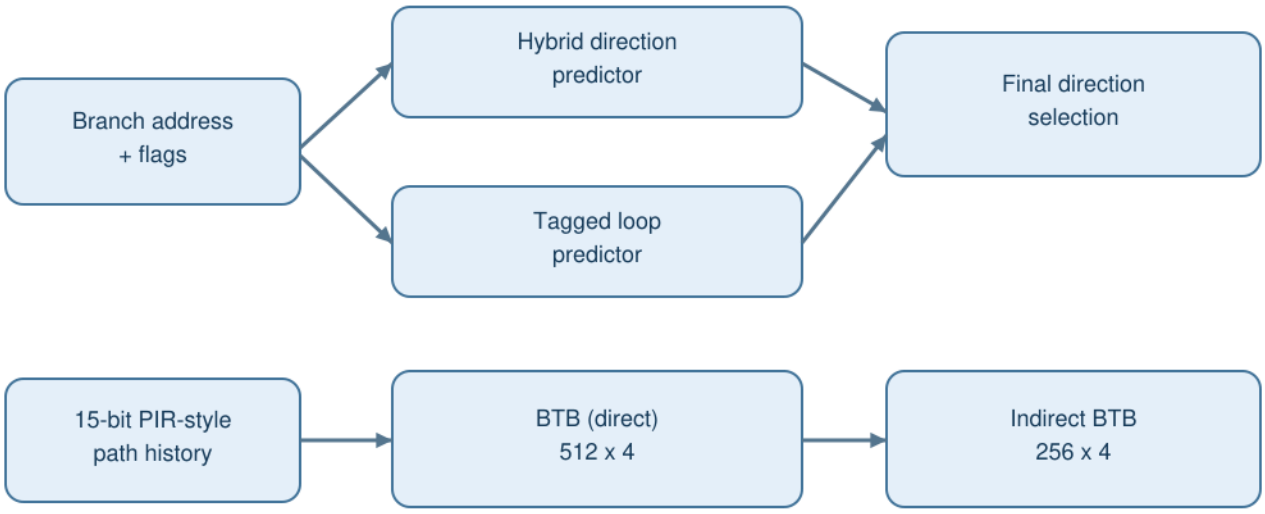
5.4 Global history update

GHR = ((GHR << 1) | (taken ? 1 : 0)) & 0x7FFF

Only resolved conditional outcomes enter the main GHR. This makes the next global lookup dependent on recent branch behavior. The simulation updates after the branch is resolved; it does not model speculative rollback of the history register.

6. Extra-credit complete-unit prototype

equivalence.



The optional cpm_predictor builds on the required direction predictor and introduces four additional mechanisms. The term “complete-unit prototype” describes the set of components included; it does not establish exact Pentium M hardware equivalence.

6.1 Loop predictor

A 64-set, two-way tagged loop table tracks an iteration count, learned loop limit, direction, confidence and replacement age. A loop entry is used to override the hybrid direction only when confidence is high and a stable trip count has been observed. This implementation is a simplified loop-exit predictor, not a verified Intel policy.

6.2 Direct and indirect target buffers

The direct BTB is a 512-set, four-way target table. The indirect BTB is a 256-set, four-way table indexed with a path-dependent hash. Both store valid tags, target addresses and LRU metadata. A taken branch updates the appropriate target entry.

6.3 PIR-style path history

A 15-bit path-information register is updated using branch-address or target information mixed with the outcome. It is used for path-sensitive lookup. The exact proprietary Pentium M PIR hashing and recovery policies are not established by the course materials.

7. Evaluation methodology and metrics

7.1 Test environment

Both predictors were compiled in the supplied C++ CBP-2 simulator environment and executed on all 20 compressed traces included with the assignment. Each run produces direction MPKI; the extra-credit executable additionally reports target MPKI. The experiment is deterministic for a fixed source version and trace.

7.2 Metric definitions

Direction MPKI is the number of incorrect branch-direction predictions per 1,000 retired instructions. Target MPKI is the number of incorrect target predictions per 1,000 instructions under the simulator's scoring rule. Lower values are better. The report uses the arithmetic mean of the 20 per-trace MPKI values, not a pooled instruction-weighted mean.

MPKI = 1000 * mispredictions / instructions 20-trace mean = (sum of per-trace MPKI values) / 20

7.3 Run protocol

The evaluation used the same trace set for the main and extra-credit executables. For each trace, the output was recorded in a CSV file with the trace name, main direction MPKI, extra-credit direction MPKI and extra-credit target MPKI. Results are rounded to three decimals in tables; averages are computed from those recorded values.

Validation layer Purpose Observed status

Compilation Check source and interface integration Passed for both executables

20-trace run Exercise complete benchmark suite 20/20 for each executable

Focused C++ assertions Check selected implementation invariants Reported passed in project workflow

Appendix B replay Compare miniature handout behavior 22/22 predictions, hits and GHR

Hardware equivalence Check exact original Intel behavior Not established

7.4 What was not measured

This project does not report wall-clock performance improvement, energy savings, physical silicon accuracy, or confidence intervals across repeated randomized trials. The simulator produces prediction-error metrics; no controlled baseline sweep or 16-configuration design-space search was run for this report.

7.5 Interpretation precautions

Two predictors can have similar direction MPKI but very different target MPKI. A predictor with more architectural blocks is not guaranteed to have better measured accuracy. It is therefore necessary to examine trace-by-trace outcomes rather than infer improvement from the extra-credit component count.

8. Full benchmark results — 20 traces

Trace	Main direction	Extra direction	Extra target
164.gzip	12.436	12.451	20.576
175.vpr	13.298	13.244	15.489
176.gcc	9.285	9.406	18.855
181.mcf	15.621	15.807	39.576
186.crafty	5.211	5.217	7.836
197.parser	8.538	8.702	17.156
201.compress	7.702	7.650	8.796
202.jess	1.118	1.097	5.833
205.raytrace	2.307	2.307	5.891
209.db	3.615	3.610	9.001
213.javac	2.037	2.074	5.363
222.mpegaudio	2.028	2.065	5.267
227.mtrt	2.383	2.383	4.116
228.jack	2.353	2.362	7.340
252.eon	1.425	1.454	8.591
253.perlbmk	1.819	1.818	6.989
254.gap	3.507	4.241	14.777
255.vortex	0.828	0.824	7.036
256.bzip2	0.057	0.057	0.172
300.twolf	21.389	21.513	26.873
Mean	5.848	5.914	11.777

The main direction predictor ranges from 0.057 MPKI on 256.bzip2 to 21.389 MPKI on 300.twolf. The extra-credit prototype follows a broadly similar pattern but has slightly higher mean direction MPKI. The target predictor has a separate error profile and should be analyzed on its own.

9.1 Overall comparison

9.2 Trace-dependent behavior

The highest main direction MPKI values occur on 300.twolf (21.389), 181.mcf (15.621), 175.vpr (13.298), and 164.gzip (12.436). Several other traces fall below 3 MPKI. This spread is consistent with different dynamic branch patterns, but without per-branch instrumentation it would be speculative to attribute each program's result to a specific control-flow structure.

9.3 Why extra credit may not improve direction

The extra-credit direction predictor may override a correct hybrid decision when loop confidence is incorrectly high, or fail to improve a difficult branch if the loop table has not learned a stable count. These are plausible design explanations, not experimentally isolated causes. Controlled ablations would be required to distinguish them.

10.1 Aggregate target results

The extra-credit prototype's unweighted mean target MPKI is 11.777. The target-error rate varies substantially across the suite. The target metric includes the behavior of the direct BTB, indirect BTB and simulator target-scoring conventions.

10.2 Why target prediction is more difficult

Direction answers only whether control flow changes. Target prediction must identify the destination, which can vary for indirect branches and can be affected by return or call behavior. A target buffer can learn previously observed destinations, but the first encounter and collisions are difficult. A path-indexed indirect table can distinguish some contexts but still aliases different histories.

10.3 Limitations of the implemented target path

The supplied branch interface does not expose all instruction-length or microarchitectural information needed for an exact x86 fall-through target predictor. The implementation uses the available branch address, branch type and resolved target. Thus the target predictor is best characterized as a functional prototype, not a cycle-accurate Pentium M reconstruction.

10.4 Appropriate extra-credit claim

The project demonstrates that the additional components were implemented, compiled and exercised on the entire suite. It does not establish that every extra-credit algorithm matches Intel's undisclosed replacement, training, hashing and speculative recovery policies. The measured results should be evaluated on that basis.

11. Appendix B worked-example verification

Verification	Matches
Predictions	22 / 22
Global hit/miss	22 / 22
GHR values	22 / 22

The handout provides a miniature predictor with 16 bimodal counters, a four-bit GHR, four global rows, two ways per row, and two-bit tags. The independent replay uses these miniature parameters and the handout's initialization and replacement conventions. It is deliberately separate from the production predictor, whose table dimensions and initialization differ.

11.1 Row-by-row reproduction

Step	Branch	GHR	Hit	Computed	Handout
1	b1	0	No	NT	NT
2	b3	1	No	NT	NT
3	b1	3	No	T	T
4	b2	6	No	T	T
5	b3	12	No	NT	NT
6	b1	9	Yes	T	T
7	b3	3	No	T	T
8	b1	7	No	T	T
9	b2	14	No	T	T
10	b3	12	Yes	T	T
11	b1	9	Yes	T	T
12	b2	2	No	NT	NT
13	b1	5	No	NT	NT
14	b2	10	Yes	NT	NT
15	b3	4	No	T	T
16	b1	9	Yes	T	T
17	b3	3	No	T	T
18	b1	7	No	NT	NT
19	b2	14	No	NT	NT
20	b3	12	Yes	T	T
21	b1	9	Yes	T	T
22	b3	3	No	T	T

12. Verification interpretation, limitations and future work

12.1 What Appendix B establishes

The replay verifies that the miniature handout rules can reproduce every listed prediction, hit/miss decision and GHR value. It provides evidence that the indexing and update conventions have been understood. It does not prove that the full-size production C++ class, which has different table sizes and initialization, reproduces these same rows when parameterized identically.

12.2 Implementation verification matrix

Global-history approximation: the main design uses outcome history instead of the original proprietary PIR mapping. Speculation: the simulator interface and implementation do not model every speculative update and recovery detail. Loop confidence: trip-count learning can be unstable for irregular loops. Target prediction: missing fall-through information limits exact target behavior. Benchmark breadth: the 20 supplied traces are the measured workload, not a universal accuracy guarantee.

12.4 Useful next experiments

To improve confidence, a parameterized C++ Appendix B harness could test the production class directly. Additional ablation runs could compare bimodal-only, gshare, hybrid without selective allocation, two-way versus four-way associativity, and extra credit with loop prediction disabled. These are recommendations only; they are not presented as completed measurements.

12.5 Research integrity

The reference student report informed the level of detail and report organization. Its benchmark baselines, design-space experiments and extra-credit results were not adopted as this project's own evidence. All numeric benchmark tables in this report are derived from the available project CSV.

13. Reproducibility, deliverables and conclusions

13.1 Project contents

cd cbp2-infrastructure-v2/src make ./predict ../traces/164.gzip/gzip.trace.bz2 ./predict_extra_credit ../traces/164.gzip/gzip.trace.bz2 python3 ../experiments/verify_appendix_b.py

The trace filenames and executable names should be checked against the exact extracted project archive before running commands. The benchmark CSV provides the complete 20-trace record; the single-trace commands above illustrate the invocation pattern.

13.3 Conclusions

The required Pentium M-inspired direction predictor combines a 4,096-entry bimodal table with a four-way tagged global predictor and a 15-bit history register. The implementation was evaluated on all 20 supplied traces and achieved an average of 5.848 direction MPKI. A separate extra-credit prototype includes loop prediction, BTB, indirect BTB and PIR-style history, achieving 5.914 direction MPKI and 11.777 target MPKI on the same suite.

An independent Appendix B replay matched all 22 rows for prediction, global hit/miss and GHR. The work therefore includes a reproducible full-suite evaluation and a handout-level worked-example check. It does not claim unperformed production-class replay or exact equivalence to proprietary Intel hardware.

References and attribution

1. University course handout, Assignment 1: Branch Predictor Implementation, including Appendix B. 2. Supplied CBP-2 simulator, predictor interface, sample gshare implementation and benchmark traces. 3. Supplied course reference material on Pentium M branch prediction. 4. Reference student report supplied by the user, consulted for organization and presentation only; no experimental data copied.

Submission note: The PDF is intended to be the sole report in the final archive. Source files, raw benchmark results and the independent Appendix B replay remain separate supporting artifacts for reproducibility.

Additional validation considerations

A useful correctness check is to compare the direction prediction before and after training a single branch repeatedly. Two-bit counters must saturate at states zero and three, and a global hit must take precedence over the bimodal fallback. A second check deliberately maps several branch-history combinations into one set to verify valid-tag comparison, invalid-way allocation and four-way LRU replacement.

The independent Appendix B replay is a miniature model configured according to the handout. Its 22 matching predictions establish consistency for that reference configuration, but the submitted C++ class uses a larger table and a different initial state. A parameterized test harness would be required to establish direct row-by-row equivalence of the production implementation.

Reproducibility and evidence checklist

Evidence item	What it demonstrates
20-trace benchmark CSV	Recorded main and extra-credit simulator outputs
Appendix B replay CSV	Computed predictions and expected handout values
C++ predictor source	Counter, lookup, update and replacement behavior
Unit test source	Focused invariants for selected prediction paths
README	Build commands, run examples and project organization

For reproducibility, the project should be built from a clean extraction and the exact benchmark commands should be executed on the supplied traces. A successful build alone does not establish numerical equivalence; measured results should be checked against the recorded CSV. The PDF is the only report, while code, test files and raw results remain separate supporting evidence.